



PRACTICAL: 03

Aim: To implement the max-heap sorting algorithm.

Algorithm:

Max-Heap sort:
Algorithm: Input: Array of elements
Output: sorted elements
Step-1: start
Step-2: for $i = n/2$ to 1, do
Step-3: $t = 2 * i$
Step-4: $largest = i$
Step-5: $x = 2 * i + 1$
Step-6: if $1 \leq n$ and $arr[i] > arr[largest]$, then do.
 $largest = i$
Step-7: if $x \leq n$ and $arr[x] > arr[largest]$, then do.
 $largest = x$
Step-8: if $largest \neq i$ then do.
Step-9: swap $arr[largest]$ and $arr[i]$
Step-10: for ($int i = size; i > 1; i--$)
Step-11: swap $arr[i]$ and $arr[i]$
Step-12: max heapify ($arr, i-1, i$)
Step-13: stop

Program / Code:

```
#include <iostream>

using namespace std;

class MaxHeap {

public:

    int arr[100], n;

    void arrayInput() {

        cout << "Enter number of elements: ";

        cin >> n;

        cout << "Enter the elements: ";
```



```
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
}

void printArray() {
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void maxheapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && arr[left] > arr[largest]) {
        largest = left;
    }
    if (right < n && arr[right] > arr[largest]) {
        largest = right;
    }
    if (largest != i) {
        swap(arr[i], arr[largest]);
        maxheapify(arr, n, largest);
    }
}

void heapsort() {
    for (int i = n / 2 - 1; i >= 0; i--) {
        maxheapify(arr, n, i);
    }
}
```



```
        for (int i = n - 1; i > 0; i--) {
            swap(arr[0], arr[i]);
            maxheapify(arr, i, 0);
        }
    }
};

int main() {
    MaxHeap mh;
    mh.arrayInput();
    cout << "\nBefore Sorting: ";
    mh.printArray();
    mh.heapsort();
    cout << "After Sorting using Heapify method: ";
    mh.printArray();
    return 0;
}
```

Output:

Time Complexity:

Time complexity:

for $(i = n/2, i > 1, i \dots) \rightarrow n$
 max heapify (arr, n, i) $\rightarrow \log n$
 for $(i = n, i-1, i-2 \dots) \rightarrow 1$
 swap (arr[i], arr[1]) $\rightarrow 1$
 max heapify (arr, i-1, 1) $\rightarrow \log n$
 $T(n) = n \times \log n + n \times 1 \times \log n$
 $T(n) = \theta(n \log n)$

Best case: $T(n) = \theta(n \log n)$
 Avg case: $\theta(n \log n)$
 worst case: $\theta(n \log n)$

Best case:
 for $i = n/2$ to 1
 max heapify (arr, n, i)
 for $i = n$ to 2
 swap (arr[i], arr[1])
 max heapify (arr, i-1, 1) $\rightarrow \log n$
 $T(n) = \theta(n) + n \times \theta(\log n)$
 $T(n) = \theta(n) + \theta(n \log n)$
 $T(n) = \theta(n \log n)$

Worst case:
 max heapify from root to lowest level of the heap.
 $\Rightarrow \log_2 n$
 max heapify() $\rightarrow \log n$
 $T(n) = n \times \log n$
 $T(n) = \theta(n \log n)$

Average case:
 for $i = n$ to 2
 swap (i) $\rightarrow 1$
 max heapify (i) $\rightarrow \log n$
 $T(n) = n \times 1 + \log n$
 $T(n) = \theta(n) + \theta(n \log n)$
 $T(n) = \theta(n \log n)$

Conclusion:

In this practical, we implemented the Max-Heap Sort algorithm successfully using the Max-Heapify method. The largest element is repeatedly placed at the end of the array, resulting in a sorted array in ascending order. The time complexity of Heap Sort is $O(n \log n)$.